

GO BASICS



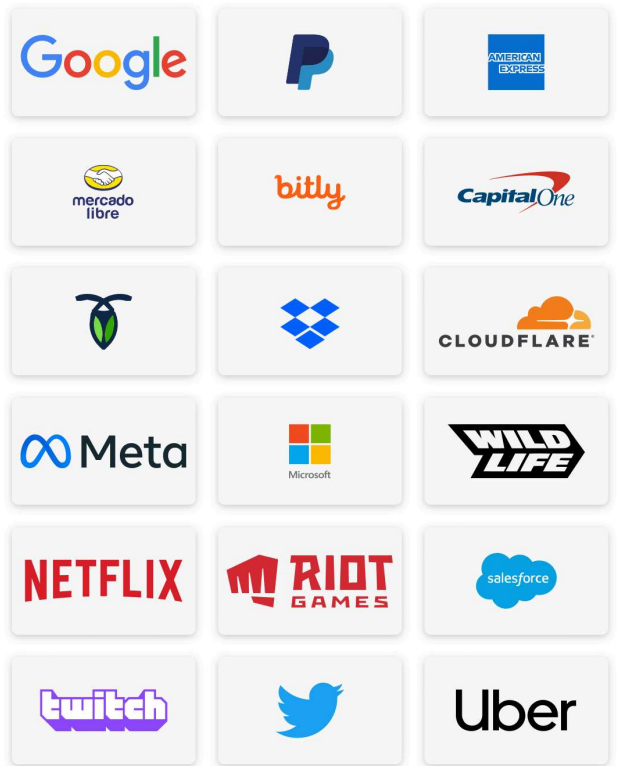
LINGUAGGIO GO

Costruire software veloce, affidabile ed efficiente su larga scala.

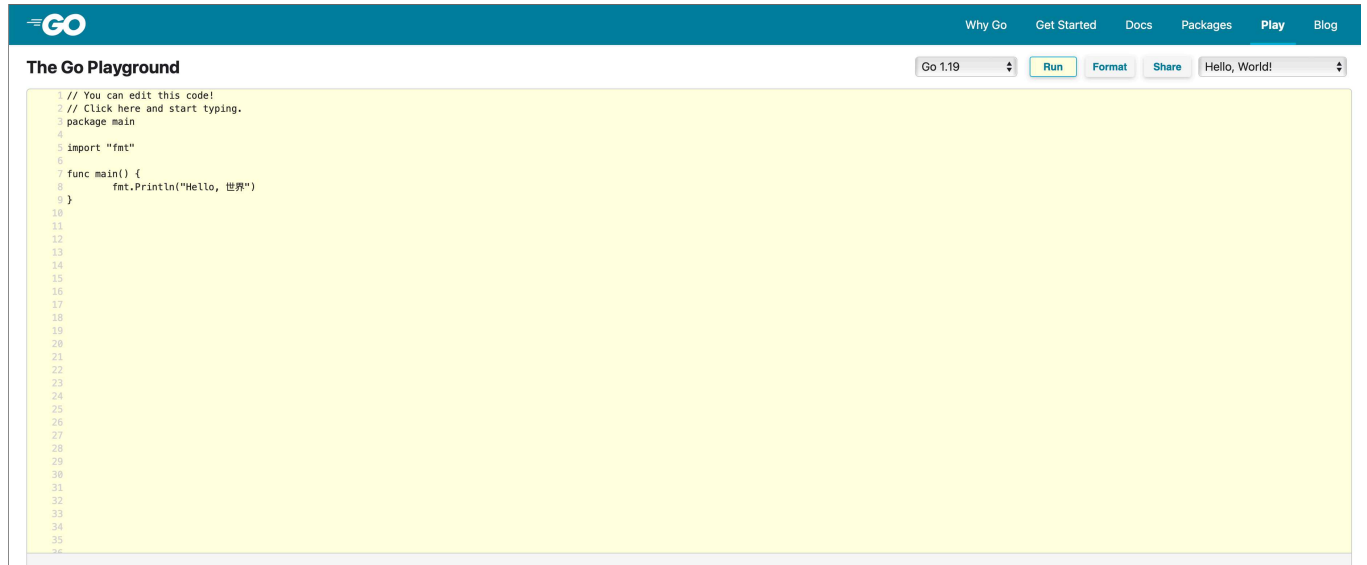
- Go è un linguaggio di programmazione **open source** supportato da Google.
- Facile da imparare e da usare.
- Concorrenza integrata e una libreria standard robusta.
- Ecosistema in crescita di partner, comunità e strumenti.



TESTIMONIAL



GO PLAYGROUND



The screenshot shows the Go Playground website. At the top is a blue navigation bar with the Go logo on the left and links for 'Why Go', 'Get Started', 'Docs', 'Packages', 'Play' (which is highlighted), and 'Blog' on the right. Below the navigation bar, the page title 'The Go Playground' is on the left. To its right is a toolbar containing a version dropdown set to 'Go 1.19', a 'Run' button, a 'Format' button, a 'Share' button, and a text input field containing 'Hello, World!'. The main area is a large yellow code editor with line numbers 1 through 36 on the left margin. The code in the editor is as follows:

```
1 // You can edit this code!
2 // Click here and start typing.
3 package main
4
5 import "fmt"
6
7 func main() {
8     fmt.Println("Hello, 世界")
9 }
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
```



<https://go.dev/play/>

PACCHETTI (PACKAGES)

- Le funzioni Go sono raggruppate in pacchetti.
- Ogni pacchetto è composto da uno o più file nella stessa directory.
- Il pacchetto è dichiarato all'inizio del file, es: `package main`.



IMPORT

Un file può importare altri pacchetti, es.:

```
import (  
    "fmt"  
    "math/rand"  
)
```



HELLO WORLD

Importiamo il pacchetto `fmt`, che contiene funzioni per la formattazione e la stampa del testo.

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

- È possibile utilizzare le funzioni del pacchetto importato se il loro nome inizia con una **lettera maiuscola**.
- Lo stesso vale per le variabili (es. `math.Pi`).

VARIABILI E COSTANTI: TIPIZZAZIONE E ASSEGNAZIONE

```
var i int
// stessa sintassi per `const` invece di `var`
var i int = 3
// stessa sintassi per `const`
var i = 3
// tipo inferito; funziona per `const` MA è senza tipo (untyped)
i := 3
// non si può usare con `const`
```

Le Costanti non possono essere dichiarate usando la sintassi :=.



TIPI DI DATI DI BASE

```
bool
string
int  int8  int16  int32  int64
uint uint8 uint16 uint32 uint64 uintptr
byte // alias per uint8
rune // rappresenta un codepoint Unicode
float32 float64
complex64 complex128
```

- I tipi **int**, **uint** e **uintptr** sono solitamente a **32 bit** sui sistemi a 32 bit e a **64 bit** sui sistemi a 64 bit.



VALORI DI DEFAULT

- 0 per i tipi numerici,
- false per il tipo booleano, e
- "" (la stringa vuota) per le stringhe.



INFERENZA DI TIPO

Senza specificare un tipo esplicito (utilizzando la sintassi `:=` o l'espressione `var` `=`), Go inferisce il tipo.

```
i := 42           // int  
f := 3.142        // float64  
g := 0.867 + 0.5i // complex128
```



COSTANTI NUMERICHE

Le costanti senza tipo (`untyped`) assumono un tipo solo quando vengono usate. Possono essere usate in contesti che richiedono tipi numerici diversi (come `int` e `float64`), purché il valore rientri nel range.



```
const (  
    untypedInt    = 1  
    // Non ha un tipo specifico (es. int o float64)  
    untypedFloat = 1.1  
    // Non ha un tipo specifico (es. float64)  
)  
  
func needInt(x int) int { return x*10 + 1 }  
func needFloat(x float64) float64 {  
    return x * 0.1  
}  
  
func main() {  
    fmt.Println(needInt(untypedInt))  
    // OK: untypedInt diventa int  
    fmt.Println(needFloat(untypedInt))  
    // OK: untypedInt diventa float64  
    fmt.Println(needInt(untypedFloat))  
    // Errore! Non si può convertire un float  
    // a int senza casting esplicito.  
}
```



CASTING (CONVERSIONE DI TIPO)

L'espressione **T(v)** converte il valore **v** al tipo **T**. È necessaria per convertire tra tipi numerici.

```
i := 42
f := float64(i) // i (int) convertito in f (float64)
u := uint(f)    // f (float64) convertito in u (uint)
```



DOMANDA



```
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    s := "Ciao, Mondo! 🌍" // La stringa contiene un'emoji
    // Istruzione A
    fmt.Println("Len (Byte):", len(s))
    // Istruzione B
    fmt.Println("Rune Count:", utf8.RuneCountInString(s))
}
```



RISPOSTA

1. **Len (Byte): 17**
2. **Rune Count: 14**





- **Rune:** C'è **un solo Rune** (valore 127757 o U+1F30D). Questo è il **carattere logico** che Go usa per l'iterazione e il conteggio.
- **Byte:** Questo singolo Rune (U+1F30D) viene codificato in UTF-8 usando **4 byte** (0xF0, 0x9F, 0x8C, 0x8D).



Categoria Rune	Intervallo Code Point (Hex)	Byte per Rune (in UTF-8)	Esempio Carattere	Spiegazione
1 Byte	U+0000 a U+007F	1 Byte	A, 1, !	Caratteri ASCII di base (inglese).
2 Byte	U+0080 a U+07FF	2 Byte	é, ñ, £	Caratteri latini estesi (accenti, tilde), valute.
3 Byte	U+0800 a U+FFFF	3 Byte	世, π, €	Caratteri CJK (cinese, giapponese, coreano), simboli matematici, la maggior parte delle valute.
☐ 4 Byte	U+10000 a U+10FFFF	4 Byte		Emoticon, pittogrammi, caratteri rari.

LOOP

- Un solo tipo: **for**
- Nessuna parentesi () dopo for
- Sempre parentesi graffe { }

```
sum := 0
for i := 0; i < 10; i++ {
    sum += i
}
```



LOOP

```
sum := 0
for i := 0; i<10; i++ {
    sum += i
}
```

```
sum := 0
i := 0 // init statement
for ; ; {
    //condition
    if i>=10 {break}

    sum += i
    i++ // post statement
}
```



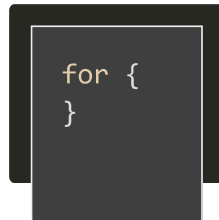
LOOP

- init and post are optional
- while

```
sum := 1
for sum < 1000 {
    sum += sum
}
```



:LOL: LOOP



IF: REGOLE E SCOPE

- Nessuna parentesi () per gli argomenti/condizioni.
- Sempre parentesi graffe { }.
- Può iniziare con un'istruzione eseguita prima della condizione (dichiarazione breve).
- Le variabili dichiarate nell'istruzione breve sono visibili solo all'interno del blocco **if** e dei successivi **else**.
- Anche gli **else if** possono avere istruzioni.



IF: ISTRUZIONE BREVE

```
// Funzione d'esempio
func pow(x, n, lim float64) float64 {
    // v è dichiarata e inizializzata qui
    if v := math.Pow(x, n); v < lim {
        return v
    }
    return lim // v non è visibile qui
}
```



SWITCH: PANORAMICA

- È una sequenza di istruzioni **if - else** ottimizzata.
- Valuta i **case** dall'alto verso il basso.
- Esegue il primo **case** la cui espressione corrisponde alla condizione dello **switch**.
- **Diverso da Java:** Ogni **case** termina **automaticamente** con un **break** implicito (non esegue i **case** successivi).
- I valori dei **case** non devono essere costanti.
- I valori non devono essere per forza interi.



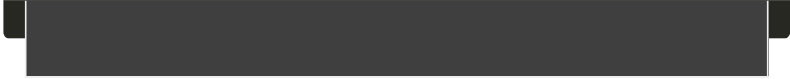
SWITCH: ESEMPIO

```
package main

import (
    "fmt"
    "runtime"
)

func main() {
    switch os := runtime.GOOS; os {
        // Istruzione breve per dichiarare 'os'
        case "darwin":
            fmt.Println("macOS.")
        case "linux":
            fmt.Println("Linux.")
        default:
            // freebsd, openbsd,
            // plan9, windows...
            fmt.Printf("%s.\n", os)
    }
}
```





DEFER (POSTICIPA)

- Posticipa l'esecuzione di una funzione fino a quando la funzione circostante non ritorna.
- Gli argomenti della funzione `defer` vengono valutati **immediatamente**, ma la chiamata alla funzione è posticipata.
- Le istruzioni `defer` vengono eseguite nell'ordine **LIFO** (Last-In, First-Out, l'ultima posticipata viene eseguita per prima).



```
package main

import "fmt"

func test() {
    defer fmt.Print(" world") // (2) Eseguita prima del ritorno di test()
    fmt.Print(" cruel")      // (1) Eseguita immediatamente
}

func main() {
    defer fmt.Println("!")    // (4) Eseguita per ultima (LIFO)
    fmt.Print("hello")        // (0) Eseguita immediatamente
    test()                   // (1) e (2)
}

// Output: hello cruel world!
```



PUNTATORI

- Contiene l'indirizzo di memoria di un valore.
- Il tipo ***T** è un puntatore a un valore di tipo T.
- Il suo valore zero è **nil**.
- **&** (operatore di indirizzo) genera un puntatore a una variabile.
- ***** (operatore di dereferenziazione) legge o imposta il valore a cui punta.
- A differenza di C, Go non ha aritmetica sui puntatori.



```
package main

import "fmt"

func main() {
    var p *int
    i := 42
    p = &i           // p punta all'indirizzo di i
    fmt.Println(p)   // stampa l'indirizzo di memoria di i
    fmt.Println(*p)  // legge il valore di i tramite il puntatore p
    *p = 21          // imposta i a 21 tramite il puntatore p
    fmt.Println(i)   // Stampa: 21
}
```



STRUCT (STRUTTURE)

- Una collezione di campi.
- Accesso ai campi tramite . (dot notation).
- **Zucchero Sintattico:** L'accesso tramite puntatore (*p).campo può essere abbreviato in p.campo.
- È possibile inizializzare solo un sottoinsieme di campi specificandoli per nome (NomeCampo:).



```
type Vertex struct {  
    X, Y int  
}  
var (  
    v1 = Vertex{1, 2}    // ha tipo Vertex  
    v2 = Vertex{X: 1}    // Y:0 è implicito  
    v3 = Vertex{}        // X:0 e Y:0 impliciti  
    p  = &Vertex{1, 2}   // ha tipo *Vertex (puntatore a Vertex)  
)
```



STRUCT ANONIME

Le struct possono essere dichiarate senza un nome di tipo esplicito se devono essere usate solo localmente.

```
a:=struct{  
    i int  
    b bool  
}{1, true}
```



ARRAY

- **`[n]T`** è un array di n valori di tipo T .
- Esempio di dichiarazione: `var a [10]int`
- Esempio di inizializzazione: `primes := [6]int{2, 3, 5, 7, 11, 13}`
- **La lunghezza è parte del tipo!** Un `[6]int` è un tipo diverso da un `[5]int`.
- **Non possono essere ridimensionati!** La dimensione è fissa.



DOMANDA (ARRAY)

- Qual è il contenuto di:

```
v := [6]int{2, 3, 5, 7}
```



Risposta: [2 3 5 7 0 0]. I valori non specificati vengono inizializzati al valore zero del tipo (0 per `int`).



SLICE

- Una vista a dimensione dinamica su un array sottostante.
- Sintassi: `a[low : high]` (l'intervallo è semi-aperto, da `low` incluso a `high` escluso).
- Tipo: `[]T` (senza specificare la dimensione).
- **Non memorizza dati direttamente**; fa riferimento a una sezione di un array.
- La modifica degli elementi di una slice **modifica l'array sottostante** e qualsiasi slice corrispondente che punta allo stesso array.



LETTERALI SLICE

- Crea un array e poi costruisce una slice che lo referencia:
- Esempio: `[]bool{true, true, false}`
- Il **valore zero** di una slice è **nil** (non ha un array sottostante).



LUNGHEZZA E CAPACITÀ DELLA SLICE

- **Lunghezza (`len`):** Il numero di elementi contenuti nella slice.
- **Capacità (`cap`):** Il numero di elementi nell'array sottostante, a partire dal primo elemento della slice.
- Espressioni usate: **`len(s)`** e **`cap(s)`**.



ESTENSIONE DELLE SLICE

- È possibile estendere una slice **verso destra** (aumentando la sua lunghezza).
- Si può estendere solo **fino alla sua capacità** massima.

```
s := []int{2, 3, 5, 7, 11, 13} // len=6, cap=6

// Affetta la slice per darle lunghezza zero.
s = s[:0] // s è [], len=0, cap=6

// Estende la sua lunghezza a 4.
s = s[:4] // s è [2 3 5 7], len=4, cap=6
```



CURIOSITÀ DELLE SLICE

- Non si può estendere una slice **verso sinistra** (non si può cambiare l'indice di partenza dopo la creazione della slice).
- Non si può indicizzare oltre la `len` se questa è inferiore a `cap`.
- Si può indicizzare oltre la `len` se si effettua un **re-slicing**.



```
a := []int{0, 1, 2, 3, 4}
// len=5, cap=5
b := a[1:4]
// b è [1 2 3], len=3, cap=4 (punta a [1, 2, 3, 4])
fmt.Println(b[3])
// Errore: l'indice 3 è fuori dalla lunghezza 3
fmt.Println(b[:cap(b)][3])
// Funziona: b[:cap(b)] crea una nuova
///slice [1 2 3 4] (len=4), l'indice 3 è valido
```



FUNZIONE MAKE

- Alloca un array azzerato e restituisce una slice che lo referencia.

```
a := make([]int, 5)    // len(a)=5, cap(a)=5. Slice [0 0 0 0 0]
```

- Per specificare una capacità diversa, si passa un terzo argomento a make:

```
b := make([]int, 0, 5) // len(b)=0, cap(b)=5. Slice [] (zero elementi, ma  
capacità 5)
```



APPEND

- **func append(s []T, vs ...T) []T**
- Il primo parametro è una slice di tipo T.
- I parametri successivi sono valori di tipo T (argomenti variabili/vararg).
- Restituisce una nuova slice con i nuovi elementi aggiunti.
- **Alloca un array più grande** se necessario (se la capacità è esaurita).



DOMANDA APPEND

```
s := []int{9000}[:0]  
s = append(s, 0)
```

- Cos'è `len(s)`?
- Cos'è `cap(s)`?
- Cosa contiene `s`?



Risposta:

- `len(s)` è 1.
- `cap(s)` è 1.
- `s` contiene `[0]`. *(Inizialmente `s` è `[9000]` con `len=1`, `cap=1`. `s[:0]` lo affetta a `[]` con `len=0`, `cap=1`. `append(s, 0)` usa la capacità esistente, impostando l'elemento 0 e restituendo una slice `[0]` con `len=1`, `cap=1`.)*



RECAP: SLICE IN GO



I. LA NATURA DELLA SLICE: ARRAY SOTTOSTANTE

Cos'è una Slice?

Una Slice è una vista dinamica (un puntatore) che referencia una sezione di un Array sottostante (l'allocazione di memoria reale).

- **Sintassi:** `a[low : high]` (da `low` incluso a `high` escluso).
- **Tipo:** `[ ]T` (senza dimensione).
- **Valore Zero:** `nil` (non referencia alcun array).

La modifica di un elemento in una slice modifica l'elemento corrispondente nell'Array sottostante e, di conseguenza, in TUTTE le altre slice che puntano allo stesso array.



2. STRUTTURA INTERNA

Ogni Slice è definita da tre componenti:

1. **Pointer (Puntatore):** L'indirizzo del primo elemento referenziato nell'array sottostante.
2. **Length (len):** Il numero di elementi nella slice.
3. **Capacity (cap):** Il numero di elementi dall'inizio della slice fino alla fine dell'array sottostante.

Si può estendere una slice ($s = s[:\text{nuova_lunghezza}]$) solo fino al valore massimo della sua cap. Non si può estendere verso sinistra.



3. ESEMPIO DI INTERDIPENDENZA (ARRAY SOTTOSTANTE CONDIVISO)

```
array_originale := [5]int{10, 20, 30, 40, 50}
// Array: [10 20 30 40 50]

// Slice A: Vista su array_originale[1:4]
slice_a := array_originale[1:4]
// Contenuto: [20 30 40], len=3, cap=4

// Slice B: Vista su array_originale[2:5]
slice_b := array_originale[2:5]
// Contenuto: [30 40 50], len=3, cap=3

// Modifica slice_a all'indice 1 (che corrisponde all'indice 2 dell'array)
slice_a[1] = 99

fmt.Println("Array modificato:", array_originale) // [10 20 99 40 50]
// slice_b (che referencia array_originale[2]) ha visto
// il suo elemento iniziale modificato!
fmt.Println("Slice B (aggiornata):", slice_b) // [99 40 50]
```

ESERCIZIO

Descrivi, riga per riga, cosa succede alla `len`, `cap` e al contenuto di `A` e `B` e la relazione con l'Array Sottostante.

```
BaseArray := [8]int{10, 20, 30, 40, 50, 60, 70, 80}
A := BaseArray[1:5]

B := BaseArray[3:6]

fmt.Printf("1. A: %v (len %d, cap %d)\n", A, len(A), cap(A))
fmt.Printf("1. B: %v (len %d, cap %d)\n", B, len(B), cap(B))

A[2] = 99

fmt.Printf("2. A: %v\n", A)
fmt.Printf("2. B: %v\n", B)

A = append(A, 100, 110, 120, 130)
```

// Fase 1: Inizializzazione

```
BaseArray := [8]int{10, 20, 30, 40, 50, 60, 70, 80}
```

```
// Slice A
A := BaseArray[1:5] // [20 30 40 50], len=4, cap=7

// Slice B, creata da una vista dell'Array sottostante di A
B := BaseArray[3:6] // [40 50 60], len=3, cap=5

fmt.Printf("1. A: %v (len %d, cap %d)\n", A, len(A), cap(A))
fmt.Printf("1. B: %v (len %d, cap %d)\n", B, len(B), cap(B))

// Fase 2: Modifica Condivisa
A[2] = 99 // Modifica l'elemento 30 a 99 nell'Array Sottostante

fmt.Printf("2. A: %v\n", A)
fmt.Printf("2. B: %v\n", B) // B deve mostrare la modifica

// Fase 3: Rilascio di un Array Sottostante
// A ha len=4 e cap=7. Aggiungiamo 4 elementi.
// L'array sottostante è troppo piccolo per ospitare 4 nuovi elementi.
// L'append DOVRÀ creare un nuovo array.
A = append(A, 100, 110, 120, 130)

fmt.Printf("3. A: %v (len %d, cap %d)\n", A, len(A), cap(A))
fmt.Printf("3. B: %v\n", B)
// B dovrebbe restare invariata (Array Sottostante NON più condiviso da A)
```

```
fmt.Printf("3. Array Base: %v\n", BaseArray)
// BaseArray deve mostrare la sola modifica di Fase 2
}
```

```
/* --- RISULTATI ATTESI ---
```

```
1. A: [20 30 40 50] (len 4, cap 7)
```

```
2. B: [40 50 60] (len 3, cap 5)
```

```
3. A: [20 30 99 50] (Correzione: A[2] è il 40, non il 30.
```

```
    L'indice 2 di A punta a BaseArray[3] che era 40)
```

```
    - [20 30 99 50] è sbagliato. A[2] è 40.
```

```
    CORRETTO Fase 2: BaseArray[3] viene modificato a 99.
```

```
2. A: [20 30 99 50]
```

```
3. B: [99 50 60] // B inizia da BaseArray[3], ora 99!
```

```
4. A: [20 30 99 50 100 110 120 130] (len 8, cap 16)
```

```
    // Append ha creato un NUOVO array.
```

```
5. B: [99 50 60] // B resta agganciato al Vecchio Array Sottostante.
```

```
6. Array Base: [10 20 30 99 50 60 70 80]
```

```
    // BaseArray mostra la sola modifica fatta da A prima dell'append.
```

```
*/
```



MAPPE

- Mappano **chiavi** a **valori**.
- Il valore zero è **nil** (e non si possono aggiungere chiavi ad una mappa **nil**).
- Si deve usare la funzione **make** per inizializzare una mappa.




```
package main

import "fmt"

var m map[string]string // m è nil, non si possono aggiungere elementi

func main() {
    m = make(map[string]string) // Inizializzazione corretta
    m["Bell Labs"] = "cooked"
    fmt.Println(m["Bell Labs"])
}
```



LETTERALI MAPPA

- Simili ai letterali struct, ma le chiavi (e i valori) sono obbligatori.

```
type Vertex struct {  
    Lat, Long float64  
}  
  
var m = map[string]Vertex{  
    "Bell Labs": Vertex{  
        40.68433, -74.39967,  
    },  
    "Google": Vertex{  
        37.42202, -122.08408,  
    },  
}
```



MODIFICARE LE MAPPE

- **Inserimento/Aggiornamento (upsert):** $m[key] = elem$
- **Lettura:** $elem := m[key]$
- **Cancellazione:** $delete(m, key)$
 - Fallisce silenziosamente se la chiave è assente.
- **Test di Presenza (due valori):** $elem, ok := m[key]$
 - **ok** è true se la chiave è presente.
 - **elem** prende il valore zero del tipo se la chiave non è presente.



RANGE

- Una forma del ciclo **for** che itera su una slice, un array, una stringa o una mappa.
- Restituisce **due valori** per iterazione: **indice** e **copia dell'elemento** (o **chiave** e **valore** per le mappe).



```
// Si itera su una slice chiamata 'pow' (esempio ipotetico)
for i, value := range pow {
    // i è l'indice, value è la copia dell'elemento
}

// Per usare solo l'indice e ignorare l'elemento (obbligatorio usare _)
for i, _ := range pow {
    // ...
}

// Forma abbreviata per solo l'indice
for i := range pow {
    // ...
}

// Per usare solo il valore e ignorare l'indice
for _, value := range pow {
    // ...
}
```



RISORSE

- **Editor:**
 - vscode + estensione golang
 - goland (usa un account educational)
- **Tutorial:**
 - go by example



CHEAT CODES



- **Importazioni Fattorizzate:**

```
import "fmt"
import "math"

// ... è uguale a:

import (
    "fmt"
    "math"
)
```



- Funziona anche per le Variabili Globali:

```
import "complex"

var (
    ToBe    bool      = false
    MaxInt   uint64     = 1<<64 - 1
    z        complex128 = complex128(complex.Sqrt(-5 + 12i))
    // Aggiunto 'complex128' e 'complex' import
)
```



- Specificare il Tipo per Parametri Multipli:

```
x int, y int // ... è uguale a:  
x, y int
```



